Rapport pour le projet

R5.10 - Nouveaux paradigmes de base de données

09/11/25

Groupe C (APP)
Axel Rivière
Quentin Tripognez

IUT Institut Universitaire Technologie LENS

UNIVERSITÉ D'ARTOIS

1. Description du projet

Ce projet consiste à mettre en place un système de recommandation de métiers à partir de compétences, en s'appuyant sur la base de connaissances européenne ESCO. ESCO fournit une classification des professions, compétences, connaissances et qualifications, organisée sous forme de graphe.

Notre but dans ce projet était de mettre en place plusieurs requêtes afin d'obtenir des informations de à partir de la base de connaissances de l'ESCO. Par exemple trouver tous les métiers pour lesquels une connaissance/compétence est requise.

Nous avons eu connaissance du sujet le 3 novembre pour un rendu le 9 novembre.

2. Tâches individuelles

Code	**Nom**
T01	Chargement des données ESCO dans Neo4j
T02	Implémentation Python Q1 - fiche métier
T03	Implémentation Python Q2 - fiche compétence/connaissance
T04	Implémentation Python Q3 - similarité entre compétences
T05	Implémentation Python Q4 - similarité entre métiers
T06	Implémentation Python Q5 - recommandation métiers
T07	Requêtes Cypher Q2 et Q4
T08	Requêtes Cypher Q1, Q3 et Q5
T09	Création de l'interface en lignes de commandes
T10	Rédaction du rapport

Table 1. Décomposition du travail du projet en tâches individuelles.

T01 : Chargement des données ESCO dans Neo4j

Cette tâche consiste en l'importation des données de l'ESCO dans Neo4j pour créer le graphe représentant la base de données.

T02 : Implémentation Python Q1 - fiche métier

Cette fonctionnalité permet d'afficher, à partir de son code, la fiche d'un métier avec toutes les compétences et connaissances associées, ainsi que ses autres appellations.

T03 : Implémentation Python Q2 - fiche compétence/connaissance

Cette fonctionnalité permet d'afficher, à partir de leur appellation, une compétence ou/et connaissance donnée où elle est requise ou facultative et pour quels métiers.

T04 : Implémentation Python Q3 - similarité entre compétences

Cette fonctionnalité permet de calculer la similarité de Jaccard entre deux compétences données tout en prenant en compte le caractère obligatoire ou facultatif de chaque compétence dans ces métiers.

T05 : Implémentation Python Q4 - similarité entre métiers

Cette fonctionnalité permet de calculer la similarité de Jaccard entre 2 métiers en se basant sur l'intersection de leurs compétences, qu'elles soient obligatoires ou supplémentaires.

T06 : Implémentation Python Q5 - recommandation métiers

Cette dernière fonctionnalité permet de retourner, à partir d'une liste de compétences, une liste des métiers à recommander en fonction de ses compétences.

T07 : Requêtes Cypher Q2 et Q4 // T08 : Requêtes Cypher Q1, Q3 et Q5

Ces deux étapes consistent à proposer des requêtes Cypher permettant de répondre aux fonctionnalités implémentées dans les tâches précédentes.

T09 : Création de l'interface en ligne de commandes

Nous avons choisit de proposer à l'utilisateur un menu permettant d'appeler les fonctions définies avec les tâches précédentes

3. Répartition des tâches

Tâche	**Membre**	**Temps**	**État**
T01	Axel Rivière	10 min	100%
T02	Axel Rivière	10 min	100%
T03	Quentin Tripognez	2h	100%
T04	Axel Rivière	2h	100%
T05	Quentin Tripognez	2h	100%

T06	Axel Rivière	2h	100%
T07	Quentin Tripognez	1h	100%
T08	Axel Rivière	1h	100%
T09	Axel et Quentin	1h	100%
T10	Axel et Quentin	3h	100%

Table 2. Répartition des tâches individuelles traitées lors du projet

3.1. Axel Rivière

J'ai d'abord commencé par intégrer et tester l'importation du fichier CSV dans *Neo4j* afin de générer le graphe de base de données.

Ensuite, j'ai travaillé sur l'implémentation de la fonctionnalité de visualisation de la fiche d'un métier. La requête utilise *MATCH* pour chercher le nœud métier correspondant au code renseigné en entrée, puis suit la relation *REQUIRES* vers les nœuds de compétences, avant de retourner le nom du métier et chaque compétence associée.

Je me suis également chargé de l'implémentation de la fonctionnalité de calcul de similarité de Jaccard entre deux compétences en analysant les métiers qui les partagent. La requête commence par rechercher les deux compétences et/ou connaissances spécifiques données en entrées, puis trouve tous les métiers qui nécessitent ou utilisent la première compétence via les relations *REQUIRES* ou *NEEDS_KNOWLEDGE*, qu'elle regroupe dans une liste *metiers_e1*. De la même manière, elle collecte tous les métiers associés à la deuxième compétence dans une liste *metiers_e2*. Elle recherche ensuite les métiers qui requièrent simultanément les deux compétences, c'est-à-dire ceux qui ont une relation avec *e1* et avec *e2*, et les stocke dans *metiers_intersection*. Les listes sont nettoyées en supprimant les valeurs nulles pour obtenir des ensembles propres *m1*, *m2* et *m_inter*. Le coefficient est alors calculé selon la formule : *Jaccard = Intersection / Union*. Cette fonction retourne trois valeurs : le nombre de métiers en intersection, le nombre de métiers dans l'union et le score de Jaccard. Un score élevé indique que les deux compétences/connaissances sont souvent requises ensemble dans les mêmes métiers.

J'ai aussi travaillé sur la fonction de recommandation de métier en fonction de compétences données. La requête commence par rechercher tous les métiers qui requièrent au moins l'une des deux compétences spécifiées, en suivant la relation *REQUIRES* entre les nœuds *Occupation* et *Skill*. Pour chaque métier trouvé, elle compte le nombre de compétences de l'utilisateur que ce métier requiert, stocké dans la variable *matching_skills* en utilisant *count(DISTINCT s)* pour éviter les doublons. Elle effectue ensuite une nouvelle recherche pour compter toutes les compétences requises par chaque métier (pas seulement celles de l'utilisateur), ce total étant stocké dans *total_skills*. La clause *WHERE matching_skills > 0* garantit qu'on ne retourne que les métiers ayant au moins une compétence en commun avec l'utilisateur. Le score est calculé avec la formule : *Score* =

Compétences correspondantes / Compétences totales du métier. Ce ratio indique le pourcentage de compétences du métier que l'utilisateur possède, un score élevé signifiant qu'il maîtrise une grande partie des compétences requises. La fonction retourne pour chaque métier son code, son nom, le nombre de compétences correspondantes, le nombre total de compétences requises et le score. Les résultats sont triés par score décroissant (*ORDER BY score DESC*) pour présenter en premier les métiers les plus adaptés au profil de l'utilisateur.

Pour finir, j'ai travaillé sur la rédaction dans le README des requêtes Cypher des questions Q1, Q3 et Q5, qui ont été récupérées des fonctions que j'ai décrites précédemment en remplaçant les variables par des données concrètes en entrées.

3.2. Quentin Tripognez

J'ai d'abord travaillé sur la fonctionnalité de recherche des métiers associés à une compétence ou une connaissance spécifique, en utilisant deux requêtes complémentaires. La première recherche tous les métiers qui nécessitent une compétence particulière : elle commence par trouver les nœuds Occupation qui ont une relation *REQUIRES* pointant vers un nœud Skill dont le nom correspond au paramètre *$name*. Pour chaque correspondance trouvée, elle retourne trois informations : le nom du métier (*o.label*), son code (*o.code*) et le type d'exigence (*r.type*) qui indique si la compétence est obligatoire ou optionnelle. Les résultats sont triés d'abord par type d'exigence, puis par nom de métier par ordre alphabétique. La seconde requête fonctionne de manière identique mais pour les connaissances : elle recherche les métiers qui ont une relation *NEEDS_KNOWLEDGE* vers un nœud Knowledge portant le nom spécifié, et retourne également le nom du métier, son code et le type d'exigence, avec le même tri appliqué. Dans le code, ces deux requêtes sont exécutées successivement puis leurs résultats sont combinés, ce qui permet de rechercher un terme sans savoir à l'avance s'il s'agit d'une compétence ou d'une connaissance, offrant ainsi une recherche plus flexible et exhaustive des métiers associés.

Ensuite, j'ai travaillé sur le calcul de la similarité de Jaccard entre deux métiers en comparant leurs compétences requises. La requête commence par rechercher le métier avec le premier code métier en entrées et suit la relation *REQUIRES* vers toutes ses compétences (Skill), en filtrant uniquement les compétences qui sont soit obligatoires (required) soit optionnelles (optional). Toutes ces compétences sont ensuite regroupées dans une liste unique *skills1* en utilisant *collect(DISTINCT s1)* pour éviter les doublons. De la même manière, elle recherche le deuxième métier et collecte toutes ses compétences requises ou optionnelles dans une liste *skills2*. Une fois les deux listes de compétences disponibles pour la comparaison, trois listes sont créées à partir des compétences collectées : *m1* (les compétences du premier métier en filtrant les valeurs nulles), *m2* (les compétences du second métier en filtrant les valeurs nulles) et *inter* (les compétences communes aux deux métiers, c'est-à-dire l'intersection). La requête calcule alors deux valeurs essentielles : l'intersection (le nombre de compétences partagées par les deux métiers) et l'union (le nombre total de compétences distinctes dans les deux métiers, calculé selon la formule : *taille de m1 + taille de m2 - taille de l'intersection*). Le score final est calculé avec la formule :

Jaccard = *Intersection / Union*. Si l'union est supérieure à zéro, le score est le ratio entre le nombre de compétences communes et le nombre total de compétences distinctes ; si l'union est nulle (cas improbable où aucun métier n'a de compétences), le score est 0. La requête retourne trois valeurs : le nombre de compétences en intersection, le nombre de compétences dans l'union et le score de Jaccard. Un score proche de 1 indique que les deux métiers sont très similaires en termes de compétences requises, tandis qu'un score proche de 0 indique qu'ils sont très différents.

Pour finir, j'ai travaillé sur la rédaction des requêtes Cypher pour les questions Q2 et Q4 qui, comme celles d'Axel, ont été récupérées des fonctions que j'ai décrites précédemment en remplaçant les variables par des données concrètes en entrées.

3.3. Axel Rivière & Quentin Tripognez

Pour finir, nous avons implémenté un menu en lignes de commandes permettant aux utilisateurs d'interagir avec les fonctionnalités, celui-ci fonctionne grâce à des input Python.

Cette tâche a été réalisée par nous deux car nous avons modifié le menu à chaque nouvelle fonctionnalité implémentée.

4. Rendu

Vous pouvez retrouver le code source du projet sur GitLab.